

ESTRUTURA DE ARQUIVOS EM C — FOLHA DE COLA

1. Bibliotecas · abrir · fechar

```
#include <stdio.h> // FILE fopen fclose fgetc fputc fread
// fwrite fseek ftell rewind feof
// printf fprintf sprintf
#include <stdlib.h> // malloc free qsort atoi
#include <string.h> // strcmp strncmp strncpy memcpy
```

```
FILE *f;
f = fopen(argv[1], "rb"); // FILE* ou NULL
if(!f){
    printf(stderr, "Arquivo %s nao pode ser aberto\n", argv[1]);
    return 1;
}
fclose(f); // nunca fclose(*f)
```

```
"rb" lê (precisa existir) "wb" escreve (cria; APAGA)
"r+b" lê e escreve "ab" escreve no FIM (não apaga)
r=read w=write a=append +=lê e escreve b=binary
```

```
saida = fopen(argv[2], "wb"); // 2º fopen falhou?
if(!saida){
    fclose(entrada); // fecha o 1º antes do return
    return 1;
}
if(argc != 2){ // nome do programa conta
    fprintf(stderr, "USO: %s [CEP]\n", argv[0]);
    return 1;
}
```

2. Ler e escrever

```
int c = fgetc(f); // int: precisa caber o EOF
fputc(c, f); // byte primeiro, arquivo depois
```

Molde: ler → testar → usar → ler de novo

```
c = fgetc(entrada);
while(c != EOF){
    /* tarefa */
    c = fgetc(entrada);
}
```

Programa	/* tarefa */	Antes do laço
Copiar	fputc(c, saida);	abrir entrada e saida
Contar linhas	if(c=='\n'){ count++; }	int count = 0;
Contar bytes	contador[c]++;	int contador[256]={0};

```
// só letras:
if(contador[i] > 0 && ((i>='A'&&i<='Z') || (i>='a'&&i<='z'))){
    printf("%c: %d\n", i, contador[i]);
}
// printf("%c",65) → A · printf("%d",65) → 65 · 1 byte = 256 valores
```

```
fread (ONDE, TAMANHO, QUANTOS, ARQUIVO) ARQUIVO → MEMÓRIA
fwrite(ORIGEM, TAMANHO, QUANTOS, ARQUIVO) MEMÓRIA → ARQUIVO
retorno = nº de ITENS (não bytes)

fread(&e, sizeof(Endereco), 1, f); // struct → &
fread(e, sizeof(Endereco), qtd, f); // ponteiro/array → sem &
if(fread(e, sizeof(Endereco), qtd, f) == qtd) printf("Lido = OK\n");
```

Copiar em blocos:

```
char buffer[8192];
qtd = fread(buffer, sizeof(char), 8192, entrada);
while(qtd > 0){
    fwrite(buffer, sizeof(char), qtd, saida); // qtd, não 8192
    qtd = fread(buffer, sizeof(char), 8192, entrada);
}
```

3. Posição no arquivo

```
fseek(f, deslocamento_em_BYTES, origem); // MOVE (não lê)
long pos = ftell(f); // INFORMA (não move)
rewind(f); // volta ao início
feof(f); // fim? só depois de uma leitura falhar
fread → LÊ E AVANÇA (600 + 300 lidos → 900)
```

Origem	Conta a partir de	Exemplo → resultado
SEEK_SET	início	fseek(f,300,SEEK_SET) → byte 300
SEEK_CUR	posição atual	fseek(f,300,SEEK_CUR) → anda 300 fseek(f,-300,SEEK_CUR) → volta 300
SEEK_END	fim	fseek(f,0,SEEK_END) → vai pro fim

```
// bytes → registros (medir o arquivo)
fseek(f, 0, SEEK_END);
long tamanhoBytes = ftell(f);
long qtd = tamanhoBytes / sizeof(Endereco);
rewind(f); // se o próximo passo for fread
// registro → bytes
fseek(f, n * sizeof(Endereco), SEEK_SET); // 499×300 = 149.700
fseek(f, -(long)sizeof(Endereco), SEEK_END);// último registro
```

4. Memória · strings · ponteiros

```
Endereco *e = (Endereco*) malloc(qtd * sizeof(Endereco));
if(e == NULL){
    fprintf(stderr, "Erro ao alocar memoria\n");
}
```

```
fclose(f); return 1;
}
free(e);
memset(p, 0, 1000 * sizeof(Pessoa)); // (ender., valor, BYTES)
Pessoa p[1000] = {0};
sizeof(Endereco) = UM registro (300), não o arquivo
FILE* nunca leva malloc/free
```

```
strcpy(indice[i].cep, e.cep, 8); // campo fixo: com tamanho
strcpy(p[0].nome, "Renato Mauro"); // array não usa =
sprintf(nome, "cep_%d.dat", i); // i=3 → "cep_3.dat"
printf("%.8s\n", e.cep); // máx. 8 (sem '\0')
int r = strcmp(a, b, 8);
// r<0 a ANTES · r==0 igual · r>0 a DEPOIS
```

```
FILE *f; f → FILE* (endereço) *f → FILE (f = fopen ✓)
Endereco e; e → struct &e → endereço
Endereco *e; e → endereço *e → struct apontada
e.campo → struct · e->campo → ponteiro · e[i].campo → bloco
```

Situação	Declaração	malloc	Campo	fread
1 por vez	Endereco e;	não	e.cep	&e
vários	Endereco *e;	sim	e[i].cep	e
arquivo	FILE *f;	nunca	—	—

5. qsort e compara

```
qsort(vetor, quantidade, sizeof(Tipo), compara); // compara SEM ()
```

Função	2º parâmetro	3º parâmetro
fread / fwrite	TAMANHO	QUANTIDADE
qsort	QUANTIDADE	TAMANHO

O sizeof do qsort e o casting do compara = **tipo do bloco** ordenado.

```
int compara(const void *e1, const void *e2){ // Endereco
    return strcmp(((Endereco*)e1)->cep, ((Endereco*)e2)->cep, 8);
}
int compara(const void *a, const void *b){ // IndiceCep
    return strcmp(((IndiceCep*)a)->cep, ((IndiceCep*)b)->cep, 8);
}
// numérico: if(x<y) return -1; if(x>y) return 1; return 0;
```

```
struct _Endereco{
    char logradouro[72]; char bairro[72];
    char cidade[72]; char uf[72];
    char sigla[2]; char cep[8];
    char lixo[2]; // 300 bytes
};
struct indiceCep{
    char cep[8];
    long posicao; // nº do registro
};
```

6. Ordenar arquivo inteiro

```
f = fopen("cep.dat", "rb");
fseek(f, 0, SEEK_END);
qtd = ftell(f) / sizeof(Endereco);
e = (Endereco*) malloc(qtd * sizeof(Endereco));
rewind(f);
fread(e, sizeof(Endereco), qtd, f); // carrega TUDO
qsort(e, qtd, sizeof(Endereco), compara);
saida = fopen("cep-ordenado.dat", "wb");
fwrite(e, sizeof(Endereco), qtd, saida);
fclose(saida); fclose(f); free(e);
```

7. Busca sequencial (sem fseek)

```
fread(&e, sizeof(Endereco), 1, f); // lê ANTES
while(!feof(f)){
    /* compara / soma / conta */
    fread(&e, sizeof(Endereco), 1, f); // lê o próximo
}
// equivale a: while(fread(&e, sizeof(Endereco), 1, f) == 1){ ... }
```

```
// NÃO ordenado:
if(strcmp(argv[1], e.cep, 8) == 0){ achou = 1; break; }
```

```
// ORDENADO: ANTES continua · IGUAL faz · DEPOIS break
int r = strcmp(argv[1], e.cep, 8);
if(r == 0){ achou = 1; break; } // repete? soma, sem break
else if(r < 0){ break; } // já passou
```

8. Busca binária (arquivo ORDENADO)

```
long inicio = 0, fim = qtd - 1;
while(inicio <= fim){
    long meio = (inicio + fim) / 2;
    fseek(f, meio * sizeof(Endereco), SEEK_SET); // sempre
    fread(&e, sizeof(Endereco), 1, f);
    int r = strcmp(argv[1], e.cep, 8);
    if(r == 0){ /* achou */ break; }
    else if(r > 0) inicio = meio + 1; // maior → direita
    else fim = meio - 1; // menor → esquerda
}
```

9. Índice (chave → posição)

```
indice = malloc(qtd * sizeof(IndiceCep));
rewind(f);
for(i = 0; i < qtd; i++){
    fread(&e, sizeof(Endereco), 1, f);
    strncpy(indice[i].cep, e.cep, 8);
    indice[i].posicao = i;
}
qsort(indice, qtd, sizeof(IndiceCep), compara);
// busca binária na MEMÓRIA: resultado = strcmp(argv[1],
indice[meio].cep, 8);
if(resultado == 0){
    fseek(f, indice[meio].posicao * sizeof(Endereco), SEEK_SET);
    fread(&e, sizeof(Endereco), 1, f);
    printf("%.72s\n%.2s\n%.8s\n", e.logradouro, e.sigla, e.cep);
    break;
}
free(indice); fclose(f);
// ordenar o índice NÃO muda posicao
```

	Sequencial	Binária	Índice
Ordenado?	não (otim.: sim)	sim	não
fseek	não	cada tentativa	só ao achar
Soma/conta?	sim	não	não

10. Ordenação externa

```
#define PARTES 8
quantidade = ftell(cep) / sizeof(Endereco); // após SEEK_END
long divisao = quantidade / PARTES;
long resto = quantidade % PARTES; // 100/8 → 12 e 4
rewind(cep);
e = (Endereco*) malloc((divisao + 1) * sizeof(Endereco));
for(int i = 0; i < PARTES; i++){
    long quantidadeParte = divisao;
    if(i < resto){ quantidadeParte++; } // resto nas primeiras
    fread(e, sizeof(Endereco), quantidadeParte, cep); // continua
    qsort(e, quantidadeParte, sizeof(Endereco), compara);
    sprintf(nome, "cep_%d.dat", i);
    FILE *parte = fopen(nome, "wb");
    fwrite(e, sizeof(Endereco), quantidadeParte, parte);
    fclose(parte);
}
fclose(cep);
```

Intercalação (lê ANTES do while):

```
void intercala(char *arqA, char *arqB, char *arqSaida){
    FILE *a, *b, *saida; Endereco ea, eb;
    a = fopen(arqA, "rb"); b = fopen(arqB, "rb");
    saida = fopen(arqSaida, "wb");
    if(!a || !b || !saida){
        fprintf(stderr, "Arquivo nao pode ser aberto.\n");
        return; // void → return sem
    }
    valor
    fread(&ea, sizeof(Endereco), 1, a);
    fread(&eb, sizeof(Endereco), 1, b);
    while(!feof(a) && !feof(b)){
        if(compara(&ea, &eb) < 0){
            fwrite(&ea, sizeof(Endereco), 1, saida);
            fread(&ea, sizeof(Endereco), 1, a);
        }else{
            fwrite(&eb, sizeof(Endereco), 1, saida);
            fread(&eb, sizeof(Endereco), 1, b);
        }
    }
    while(!feof(a)){ // resto de A
        fwrite(&ea, sizeof(Endereco), 1, saida);
        fread(&ea, sizeof(Endereco), 1, a);
    }
    while(!feof(b)){ // resto de B
        fwrite(&eb, sizeof(Endereco), 1, saida);
        fread(&eb, sizeof(Endereco), 1, b);
    }
    fclose(a); fclose(b); fclose(saida);
}
```

ERRADO: while(!feof(f)){ fread; fwrite; } → testa antes de ler → duplica o último registro

Chamadas (PARTES – 1 = 7 intercalações):

```
int arquivoNome = PARTES;
for(int i = 0; i < (PARTES - 1) * 2; i += 2){
    char nome1[20], nome2[20], nome3[20];
    sprintf(nome1, "cep_%d.dat", i);
    sprintf(nome2, "cep_%d.dat", i + 1);
    sprintf(nome3, "cep_%d.dat", arquivoNome); // ou PARTES + i/2
    intercala(nome1, nome2, nome3);
    arquivoNome++;
}
free(e);
```

i=0 cep_0+cep_1 → cep_8

i=2 cep_2+cep_3 → cep_9

i=4 cep_4+cep_5 → cep_10

i=6 cep_6+cep_7 → cep_11

i=8 cep_8+cep_9 → cep_12

i=10 cep_10+cep_11 → cep_13

i=12 cep_12+cep_13 → cep_14 * final

(8-1)*2 = 14 → i = 0..12 (7 voltas)

quantidadeIntercala = PARTES – 1 → usar quantidadeIntercala * 2. Não subtrair o 1 duas vezes!

11. Lógica pelo enunciado

Enunciado fala...	Pensar em...
tamanho / nº de registros	SEEK_END + ftell + sizeof
leia um por um / até encontrar	sequencial + break ao achar
ordenado + pare após passar	sequencial otimizada
localize 1 registro + ordenado	binária pode ser usada
meio / metade / posição central	binária + fseek
calcule total / quantos	percorrer e acumular
para cada ano · maior e menor de cada ano	quebra de grupo
adicione ao final / inclusão em lote	fopen(..., "ab") + copiar
posicione no n-ésimo	fseek(f, (n-1)*sizeof(T), SEEK_SET)
avance/retroceda da posição atual	SEEK_CUR (o fread já avançou!)
último registro	fseek(f, -(long)sizeof(T), SEEK_END)
lacunas / dias ausentes	atual != anterior + 1
média móvel / janela k	lê k, volta k–1 (SEEK_CUR)
2 arquivos ordenados → 1	intercalação (igual → soma, avança os 2)
não leia o arquivo todo	1 registro/bloco fixo, sem malloc(qtd)

Quebra de grupo (maior/menor por ano):

```
fread(&p, sizeof(Pagamento), 1, f);
anoAtual = p.ano; maior = p; menor = p; // grupo começa no 1º
fread(&p, sizeof(Pagamento), 1, f);
while(!feof(f)){
    if(p.ano != anoAtual){ // NOVO ANO → fecha
        imprime(maior); imprime(menor);
        anoAtual = p.ano; maior = p; menor = p;
    }else{ // MESMO ANO
        if(p.valor > maior.valor) maior = p;
        if(p.valor < menor.valor) menor = p;
    }
    fread(&p, sizeof(Pagamento), 1, f);
}
imprime(maior); imprime(menor); // ATENÇÃO: último ano
```

Total do período (ordenado por ano e mês):

```
int alvo = ano * 100 + mes; // 03/2024 → 202403
int atual = p.ano * 100 + p.mes;
if(atual == alvo) total += p.valor; // IGUAL: soma e continua
else if(atual > alvo) break; // DEPOIS: para
```

Lacuna · janela móvel · intercalação com soma:

```
if(r.dia != anterior + 1) printf("faltam %d a %d\n", anterior+1, r.dia-1);
anterior = r.dia; // atualiza sempre

while(fread(bloco, sizeof(Registro), k, f) == k){
    /* média do bloco · dia = bloco[k-1].dia · fwrite */
    fseek(f, -(long)((k - 1) * sizeof(Registro)), SEEK_CUR);
}

if(ra.dia < rb.dia){ fwrite(&ra,..); fread(&ra,..,a); }
else if(ra.dia > rb.dia){ fwrite(&rb,..); fread(&rb,..,b); }
else{ soma = ra; soma.novos_casos += rb.novos_casos;
      soma.obitos += rb.obitos; fwrite(&soma,..);
      fread(&ra,..,a); fread(&rb,..,b); } // avança OS DOIS
```

Navegação com fseek (1º registro = índice 0):

5º → fseek(f, 4*sizeof(T), SEEK_SET) · após o fread a posição vai p/ 6º

8º → fseek(f, 7*sizeof(T), SEEK_SET) · após o fread está no índice 8

6º a partir daí → fseek(f, -3*(long)sizeof(T), SEEK_CUR) (-2 cai no 7º!)

último → fseek(f, -(long)sizeof(T), SEEK_END)

12. Ideia central e lembretes

Registros têm TAMANHO FIXO → BYTES ↔ REGISTROS

qtd de registros = tamanho do arquivo ÷ sizeof(T)

posição em bytes = nº do registro × sizeof(T)

- fseek → acesso direto → busca binária · índice
- contar e dividir registros → ordenação externa (sem fseek)

PONTEIROS

f → endereço do FILE · *f → FILE · &e → endereço
· → struct · -> → ponteiro para struct

ARQUIVO

fseek MOVE (ONDE) · fread LÊ E AVANÇA (O QUE)
ftell INFORMA · rewind VOLTA AO INÍCIO
SEEK SET início · SEEK_CUR atual · SEEK_END fim
fgetc 1 byte · fread itens · int c (EOF)
ler → testar → usar → ler de novo
rewind depois do SEEK_END (se for dar fread)

CONTAS

BYTES ÷ sizeof → REG · REG × sizeof → BYTES

MEMÓRIA

malloc reserva · free libera · testar NULL
qsort: QTD antes do TAMANHO (fread é o contrário)

PROGRAMAS

binária → ordenado + fseek a cada tentativa
índice → chave+posição → ordena → busca → fseek
externa → divide → qsort → intercala 2 a 2 → 1 arquivo
PARTES 8 → 7 intercalações → final cep_14 (2×PARTES–2)
ANTES continua · IGUAL faz · DEPOIS break
para cada X → mudou? fecha grupo · imprime o último no fim

LÓGICA

ano e mês → ano*100+mes · adicionar → "ab"
depois do fread a posição já está no próximo registro